

Public	OBJECTIF PÉDAGOGIQUE
Bach2 / ING1	OBJECTIF PÉDAGOGIQUE : Comprendre l'infrastructure d'hébergement moderne, les bases des réseaux IP et les principes de la conteneurisation. Être capable d'automatiser le déploiement continu d'une application, de sécuriser les échanges et de superviser un service en production.
Durée	
12 heures	
Prérequis	
Linux et Bash B2 Notions de développement web	Objectifs Opérationnels : À la fin du module, les apprenants seront capables de : • Expliquer l'adressage IP, le sous-réseau, le DNS et le routage • Distinguer IaaS, PaaS, SaaS et hébergement Serverless • Conteneuriser une application avec Docker • Construire un pipeline CI/CD automatisé • Sécuriser les échanges avec SSL/TLS et gérer les secrets • Analyser les logs et mettre en place un monitoring de base • Auditer un fichier de configuration généré par IA
Modalités d'évaluation	
TP : pipeline CI/CD et application déployée en ligne	
Modalités pédagogiques	
Cours TP sur machine Démonstrations	PROGRAMME FONDAMENTAUX RÉSEAU • Modèle en couches et encapsulation des données • Adressage IPv4 et IPv6 : structure et notation • Masques de sous-réseau et découpage CIDR • Résolution de noms : DNS, enregistrements A, CNAME, MX • Routage, passerelle par défaut et outils de diagnostic CLOUD COMPUTING • Définition du Cloud : élasticité, mutualisation, à la demande • IaaS, PaaS et SaaS : périmètres et responsabilités • Hébergement Serverless : Vercel, Netlify, fonctions cloud • Modèles de coût et pièges de facturation • Choisir un mode d'hébergement selon le projet CONTENEURISATION AVEC DOCKER • Machine virtuelle et conteneur : différences et usages • Images, conteneurs, couches et registre • Rédaction d'un Dockerfile : instructions essentielles • Volumes et persistance des données • Docker Compose : orchestrer plusieurs services PHILOSOPHIE DEVOPS • Origine du DevOps : casser le mur entre Dev et Ops • Intégration continue : automatiser build et tests • Déploiement continu : livrer petit et souvent • Infrastructure as Code : principe et bénéfices • Indicateurs DORA : fréquence, délai, taux d'échec, MTTR
Matériel et logiciel	
PC + Docker GitHub Actions Vercel / Netlify	
Modalités particulières	
Classe Inversée	
Oui Non	

► Compétences (*)	PROGRAMME (SUITE)
Compétence : Conteneuriser et déployer une application via un pipeline CI/CD Critère : URL publique fonctionnelle livrée	PIPELINE CI/CD - Anatomie d'un workflow : déclencheurs, jobs, étapes - GitHub Actions : syntaxe YAML et runners - Étapes types : installation, tests, build, déploiement - Conditions, matrices et exécution parallèle - TP : pipeline complet sur un dépôt Git SÉCURITÉ DES ÉCHANGES ET DES SECRETS - HTTPS : chiffrement, intégrité, authentification du serveur - Certificats SSL/TLS : chaîne de confiance et Let's Encrypt - Renouvellement automatique et erreurs courantes - Variables d'environnement et secrets de pipeline - Ne jamais versionner un secret : bonnes pratiques MONITORING ET OBSERVABILITÉ - Trois piliers : logs, métriques, traces - Lire et filtrer les logs d'un serveur applicatif - Métriques essentielles : CPU, mémoire, temps de réponse - Alerting : seuils et destinataires - Dagnostic d'un incident à partir des journaux DÉPLOIEMENT ET VERSIONNAGE - Stratégies : recreate, rolling, blue-green, canary - Rollback : conditions de déclenchement et procédure - Versionnage sémantique et étiquetage des releases - Environnements : développement, préproduction, production - Feature flags : livrer sans activer IA APPLIQUÉE À LA CONFIGURATION ET ÉVALUATION - Générer un Dockerfile ou un workflow CI/CD avec un LLM - Auditer la configuration produite : sécurité, image de base - Repérer les mauvaises pratiques suggérées par l'IA - TP noté : conteneuriser et déployer une application - Livrable : dépôt Git, pipeline vert et URL publique
► Niveau visé	
Intermédiaire	
► Outils utilisés	
Docker GitHub Actions Vercel / Netlify	
► Contrôle des acquis	
TP pipeline CI/CD et déploiement	

(*) Compétence du référentiel et critère d'évaluation correspondant